

Capítulo 19: Programación de Datos Estructurados con Spark

1. Explicación Teórica: DataFrames, Datasets y el Optimizador Catalyst

La **Programación de Datos Estructurados** en Apache Spark se realiza predominantemente a través de las abstracciones de **DataFrame** y **Dataset**. Estas estructuras son fundamentales para el procesamiento de Big Data, ya que permiten que el motor de optimización de Spark, llamado **Catalyst**, genere planes de ejecución altamente eficientes.

DataFrames (Datasets sin tipo)

Un **DataFrame** es esencialmente un `Dataset` de tipo `Row` (`Dataset[Row]`). Proporciona una vista estructurada de los datos, similar a una tabla en una base de datos relacional. Las operaciones en DataFrames son llamadas "transformaciones sin tipo" (*untyped transformations*) porque las comprobaciones de nombres de columnas y esquemas se realizan en **tiempo de ejecución** (*runtime*). Esto ofrece flexibilidad y concisión, pero con menor seguridad de tipos.

Datasets (Datasets fuertemente tipados)

El **Dataset** es la abstracción predilecta en Scala. Representa una colección distribuida de objetos fuertemente tipados. Los Datasets son la unión perfecta entre la eficiencia de los DataFrames y la **seguridad de tipos y la programación orientada a objetos de Scala**.

Para definir el esquema de un Dataset de forma segura en tipos, se utiliza la `case class` de Scala. Spark utiliza un mecanismo llamado **Encoder** para serializar y deserializar estos objetos de Scala. Los *Encoders* están codificados dinámicamente y utilizan un formato que permite a Spark realizar operaciones clave (como filtrado, ordenación y *hashing*) sin necesidad de deserializar completamente los bytes de vuelta a objetos en cada paso, lo que resulta en una abstracción de alto rendimiento.

2. Sintaxis y Operaciones Fundamentales

La manipulación de DataFrames y Datasets se basa en una API expresiva que se traduce en optimizaciones del motor Catalyst. Para empezar a trabajar con DataFrames o Datasets, es crucial tener una instancia de `SparkSession` e importar las conversiones implícitas (los *Encoders*).

Requisito de Importación:

```

1 import spark.implicits._
2 // Habilita los Encoders y la notación de columna $"

```

A. Operaciones sin Tipo (DataFrames / Dataset[Row])

Estas operaciones manipulan los datos haciendo referencia a las columnas mediante *strings* (cadenas de texto) o notación `$`.

Operación	Descripción	Sintaxis (DataFrame)
Selección	Selecciona columnas específicas.	<code>df.select("nombre", "edad")</code>
Expresiones	Ejecuta cálculos o transformaciones en las columnas.	<code>df.select(\$"name", \$"age" + 1)</code>
Filtrado	Retiene filas que cumplen una condición (<i>predicate</i>).	<code>df.filter(\$"age" > 21)</code>
Agregación	Agrupar por una columna y aplicar una función de agregación.	<code>df.groupBy("age").count()</code>

B. Operaciones con Tipo (Datasets / Dataset[T])

Cuando se utiliza un `Dataset[T]` (donde T es una *case class*), se puede acceder a los campos directamente utilizando la sintaxis de programación funcional de Scala (lambdas y HOFs como `map` y `filter`).

Operación	Descripción	Sintaxis (Dataset[T])
Filtrado	Usa lambdas sobre el tipo T.	<code>ds.filter(p => p.edad > 21)</code>
Transformación	Crea un nuevo objeto T inmutable por cada elemento.	<code>ds.map(p => p.copy(edad = p.edad + 1))</code>

3. Ejemplos de Código Realistas

Utilizaremos la modelización de un equipo deportivo de Spark, mostrando cómo las uniones (*joins*) son esenciales en la ingeniería de datos.

```

1 import org.apache.spark.sql.{SparkSession, Dataset, Row}
2 import org.apache.spark.sql.types._
3 import org.apache.spark.sql.functions._
4
5 // 1. Inicialización de Spark (asumiendo sparkSession ya existe)
6 val spark: SparkSession = ???
7 import spark.implicits._ // Habilita Encoders y la notación $"columna"

```

```

8
9 // 2. Definición del Esquema (Usando Case Class para seguridad de tipos)
10 case class Kid(id: Int, nombre: String, teamId: Int)
11 case class Team(id: Int, nombreEquipo: String, capitanId: Int)
12
13 // 3. Creación de Datasets de ejemplo (Simulación de datos)
14 val kidsDS: Dataset[Kid] = Seq(
15     Kid(1, "Ana", 10),
16     Kid(2, "Juan", 20),
17     Kid(3, "Eva", 10),
18     Kid(4, "Leo", 99) // Kid sin equipo (ID 99 no existe)
19 ).toDS()
20
21 val teamsDS: Dataset[Team] = Seq(
22     Team(10, "Los Invencibles", 1),
23     Team(20, "Estrellas de Rock", 2)
24 ).toDS()
25
26 // --- OPERACIONES FUNDAMENTALES ---
27
28 // A. Filtrado y Agregación (Typed – Seguro en Tipos)
29 println("--- Agregación: Conteo de Miembros por Equipo ---")
30 val conteo = kidsDS
31     .groupBy(_.teamId) // Agrupa usando el campo de la case class
32     .count()           // Cuenta los miembros en cada grupo
33 conteo.show()
34 // Resultado: (teamId=10, count=2), (teamId=20, count=1), (teamId=99,
35 // count=1)
36
37 // B. Unión de DataFrames (Inner Join)
38 // Queremos la lista de niños con el nombre de su equipo.
39 // Transformamos a DataFrame (Dataset[Row]) para usar la API de columna en
40 // la unión.
41 val condicionUnion = kidsDS("teamId") === teamsDS("id") // Columna de Kid
42 // === Columna de Team
43
44 println("--- Inner Join: Solo niños con equipo existente ---")
45 val kidsTeamsDF = kidsDS.toDF() // Convertir a DataFrame (Dataset[Row])
46     .join(teamsDS.toDF(), condicionUnion, "inner")
47     .select($"nombre" as "Nombre_Niño", $"nombreEquipo")
48
49 kidsTeamsDF.show()
50 /*
51 +-----+-----+
52 |Nombre_Niño|nombreEquipo|
53 +-----+-----+

```

```

51 |Ana          |Los Invencibles|
52 |Eva          |Los Invencibles|
53 |Juan         |Estrellas de Rock|
54 +-----+-----+
55 */
56
57 // C. Unión Externa Izquierda (Left Outer Join)
58 // Queremos todos los niños, incluso si no tienen equipo (usando Left
59 // Outer Join)
60 println("--- Left Outer Join: Incluyendo al niño sin equipo ---")
61 val allKidsTeamsDF = kidsDS.toDF()
62   .join(teamsDS.toDF(), condicionUnion, "left_outer")
63   .select($"nombre" as "Nombre_Niño", $"nombreEquipo")
64
65 allKidsTeamsDF.show()
66 /*
67 +-----+-----+
68 |Nombre_Niño|nombreEquipo|
69 +-----+-----+
70 |Ana        |Los Invencibles|
71 |Eva        |Los Invencibles|
72 |Juan       |Estrellas de Rock|
73 |Leo        |null           | // Leo no tiene equipo (null)
74 +-----+-----+
75 */
76
77 // D. Anti-Join (Patrón de Filtrado)
78 // Queremos encontrar solo a los niños que NO están en un equipo.
79 println("--- Anti-Join: Niños sin equipo ---")
80 // Left Anti Join devuelve las filas del lado izquierdo para las que NO
81 // hay una coincidencia en el derecho.
82 val lonelyKidsDF = kidsDS.toDF()
83   .join(teamsDS.toDF(), condicionUnion, "left_anti")
84   .select($"nombre" as "Nombre_Niño")
85
86 lonelyKidsDF.show()

```

4. Comparativa con Java y Python

La principal ventaja de usar Scala para la programación de datos estructurados con Spark reside en la robustez de su sistema de tipos en tiempo de compilación.

Característica	Java (API de Spark)	Python (PySpark)	Scala (API de Spark)
Estructura Preferida	<code>Dataset [Row]</code> (<code>DataFrame</code>) o <code>Dataset [JavaBean]</code> .	<code>DataFrame</code> (dinámico).	<code>Dataset [CaseClass]</code> .
Seguridad del Esquema	Estático, pero requiere <i>JavaBeans</i> (más verboso que <code>case class</code>).	Dinámico. Los errores de esquema (ej. escribir mal el nombre de una columna) se detectan en <i>runtime</i> .	Estático. El compilador verifica que la <code>case class</code> y la estructura de datos coincidan en tiempo de compilación .
Expresividad en HOFs	Requiere clases anónimas o <i>lambdas</i> con <code>MapFunction</code> y <code>Encoder</code> explícito.	Usa <i>lambdas</i> (<code>lambda x: ...</code>) y sintaxis de Python.	Conciso y funcional. Utiliza directamente <i>lambdas</i> de Scala (ej. <code>.filter(_ . edad > 21)</code>).
UDFs (Lógica de Negocio)	Nativo JVM.	Penalización de rendimiento debido a la serialización de datos entre la JVM y el proceso Python.	Nativo JVM. Rendimiento superior para la lógica compleja.

5. Best Practices (*The Scala Way*)

La forma idiomática de manipular datos estructurados en Spark con Scala se basa en la seguridad de tipos y la composición funcional:

- Modelar Siempre con `case class` y `Dataset [T]`** : La mejor práctica es utilizar `case class` para definir el esquema de los datos y trabajar con `Dataset [T]` . Esto traslada la detección de errores de esquema (ej. columna inexistente) del *runtime* al **tiempo de compilación**, lo cual es un sello distintivo de la ingeniería de datos robusta en Scala.
- Usar `spark.implicits._`** : Esta importación es necesaria para obtener los `Encoder` necesarios para que Spark pueda mapear las `case class` hacia y desde el formato interno optimizado.
- Priorizar la API Columnar y Funciones Integradas:** Siempre que sea posible, utiliza las funciones optimizadas de la API columnar de Spark (ej. `"columna" + 1` , `groupBy`) o las funciones integradas de `org.apache.spark.sql.functions` (ej. `count` , `sum`). Estas funciones son procesadas por el optimizador Catalyst, que genera el *bytecode* más

eficiente. **Evita las UDFs** (User Defined Functions) a menos que la lógica no pueda expresarse de otra manera, ya que las UDFs suelen ser menos optimizables.

4. **Composición con HOFs y `for-comprehensions`** : Utiliza `map`, `filter`, `flatMap` o las *for-comprehensions* (que son azúcar sintáctico para los monads subyacentes) para aplicar transformaciones a los Datasets. Este estilo declarativo maximiza la legibilidad y la capacidad de optimización del código.

La programación de datos estructurados en Scala es como construir una base de datos distribuida con un **tipado estricto**. Al definir el esquema con `case class` antes de procesar un terabyte de datos, le das al compilador un plano arquitectónico (el `Dataset[T]`), permitiendo que el optimizador de Spark (Catalyst) construya la tubería de procesamiento más rápida posible, garantizando que si el código compila, es menos propenso a fallar a gran escala.